

Project Title Solar System Interactive Simulation

Course Computer Grafics

Instructor Dr.Mahmoud Emam

University Egyptian Chinese University

Date April 2026

NO.	NAME	ID	TASK
1	Abdelrahman Alaa	692400426	Project Manager + Core System
2	Eyad Haitham	692400171	Sun & Shader
3	Ahmed Belal	692400167	Planet Materials & Textures
4	Ahmed Mohamed	692400185	Camera & Interaction
5	Malak Emad	692400649	Mercurey Simulation
6	Nourhan Hamed	692400663	Jupiter Simulation
7	Merna Moustafa	692400177	UI System
8	Manna Elsaied	692400764	Audio & Data
9	Malak Ramy	692400721	Extra Features & Astronaut

Project Description

Solar System Interactive Simulation is a real-time 3D interactive application built using Unity 6 with the Universal Render Pipeline (URP). The project simulates all eight planets of our solar system orbiting the Sun with scientifically accurate relative speeds, axial tilts, and orbital radii.

Players can freely explore the solar system using a free-roam camera or switch to a third-person astronaut character that floats and swims through space. Clicking on any planet focuses the camera on it and displays a detailed information panel containing real scientific data including diameter, distance from Sun, orbital period, temperature, gravity, atmosphere composition , and a fun fact. Each planet also plays a unique NASA ambient sound when visited.

The simulation includes a time control slider to pause or accelerate the simulation, planet trail renderers , a procedurally generated asteroid belt, an animated comet with an elliptical orbit, and a speed of light pulse visualizer. Two dedicated simulation scenes for Mercury and Jupiter provide close-up immersive experiences of those planets.

Objectives

Write these points:

Primary Objectives:

- Build a visually realistic 3D solar system using Unity 6 and Universal Render Pipeline
- Implement real-time planet orbit and rotation using mathematical trigonometric calculations
- Create interactive planet selection with scientific data display using ScriptableObject data architecture
- Develop a custom atmosphere rim-lighting shader for realistic planetary glow effects
- Implement procedural mesh generation for Saturn's ring system
- Build a complete audio system with NASA recordings and smooth audio crossfading
- Create an immersive third-person astronaut character with physics-based floating movement
- Design dedicated simulation scenes for Mercury and Jupiter with unique visual effects

Secondary Objectives:

- Implement simulation speed control
- Add visual effects including trails, comet , asteroid belt, and speed of light visualizer
- Create an animated main menu with video intro
- Package the project as a standalone Windows .exe

System Design

Write this section with this diagram description:

Architecture Overview:

The project follows a Manager-based architecture where central manager classes coordinate between subsystems.

Scene Flow:

MainMenu → (video intro) → SolarSystem

SolarSystem → (click Enter Simulation) → Planet_Simulation

Planet_Simulation → (click Back) → SolarSystem

Core Systems:

[SolarSystemManager] ← central brain



Planet Hierarchy in Scene:

]Planet Parent[

- | PlanetMesh (sphere + material)
- | Atmosphere (custom shader)
- | Clouds (transparent sphere)
- └ Rings (Saturn only - procedural mesh)

Data Flow:

PlanetData (ScriptableObject)

- PlanetClickHandler detects click
- SolarSystemManager.FocusPlanet()
- CameraController.FocusOn()
- UIManager.ShowPlanetInfo()
- AudioManager.PlayPlanetSound()



Used Technologies

Technology	Version	Purpose
Unity	6.0.0.70f1	Game engine & rendering
Universal Render Pipeline	URP 17	Advanced rendering pipeline
C#	.NET 6	All game logic scripts
Shader Graph	URP	Sun animated shader
HLSL	Custom	Atmosphere rim shader
TextMeshPro	3.x	UI text rendering
Orbitron Font	Google	Space-themed UI font
VideoPlayer	Unity API	Main menu intro video
ScriptableObjects	Unity API	Planet data storage
Physics.Raycast	Unity API	Planet click detection
LineRenderer	Unity API	Orbit path visualization
Trail Renderer	Unity API	Planet movement trails
Particle System	Unity API	Corona, stars, asteroids
ProceduralMesh	Custom	Saturn ring generation
NASA Audio Files	Public	Planet ambient sounds
Solar System Scope	Free Pack	Planet texture maps (8K)
Poly Haven HDRI	Free	Milky Way skybox
Stylized Astronaut	Asset Store	Character model

Explanation of Implemented Features

Feature 1 — Real-time Planet Orbits

Each planet moves around the Sun using trigonometric calculations in the PlanetOrbit script. The angle increases each frame multiplied by orbit speed and the global TimeScale value. X and Z positions are calculated using Cos and Sin of the current angle multiplied by the orbit radius. The orbit path is drawn using a Line Renderer with 128 segments.

Feature 2 — Custom Sun Shader

The Sun uses a Shader Graph material with two animated fire texture layers moving in opposite directions using Time and Position nodes. Triplanar mapping samples the texture from all three axes. Simple Noise adds surface detail. The output connects to both Base Color and Emission with intensity 2.5 to create the glowing effect combined with Bloom post-processing.

Feature 3 — Atmosphere Shader

A custom HLSL shader calculates the dot product between the surface normal and the camera view direction. When the angle approaches 90 degrees at the planet edge, the rim value increases, creating a glowing atmosphere effect. Each planet has a unique atmosphere color assigned as a material property.

Feature 4 — Planet Information System

Planet data is stored in ScriptableObject assets with 10 data fields per planet. When a planet is clicked, Physics.Raycast detects the hit, calls SolarSystemManager.FocusPlanet(), which tells UIManager to populate all text fields from the ScriptableObject and show the panel. A Simulation button uses SceneLoader to transition to the planet's dedicated scene.

Feature 5 — Saturn Rings (Procedural Mesh)

The RingGenerator script creates a flat ring mesh at runtime. It loops through 128 segments, calculating inner and outer vertex positions using polar coordinates. Triangles are constructed between adjacent inner and outer vertices to form the ring surface. A transparent material with the Saturn rings texture is applied.

Feature 6 — Audio System

AudioManager maintains two AudioSource components. The background music plays immediately on scene load. When a planet is focused, a Coroutine gradually reduces background volume to zero while the planet ambient sound fades in. Pressing Escape reverses this process. All NASA sounds are imported with Vorbis compression.

Feature 7 — Astronaut Character

The third-person astronaut uses the Stylized Astronaut asset from the Unity Asset Store. The AstronautPlayer script handles WASD movement in 3D space including Q/E for vertical movement. A floating bob effect uses Mathf.Sin on time to create weightless oscillation. When within 15 units of a planet, a button appears and pressing Enter triggers planet details mode.

Feature 8 — Time Control System

A UI Slider sets the static `TimeController.TimeScale` value between 0 and 10. `PlanetOrbit` and `PlanetRotation` multiply their movement speeds by this value every frame. Setting it to 0 pauses all movement. Quick buttons for pause, normal, and fast-forward provide convenient shortcuts.

Feature 9 — Visual Effects

`Planet Trail Renderers` leave colored streaks behind each planet as they orbit. The Asteroid Belt uses a Particle System with 200 particles on a circular shape at radius 66 (between Mars and Jupiter). The Comet uses an elliptical orbit script with a Trail Renderer and particle tail. The Speed of Light visualizer spawns expanding ring meshes from the Sun every 3 seconds.

Feature 10 — Mercury Simulation

[Team Member 8 writes this — describe what their scene does: temperature shader, crater dust particles, day/night visualization, UI data]

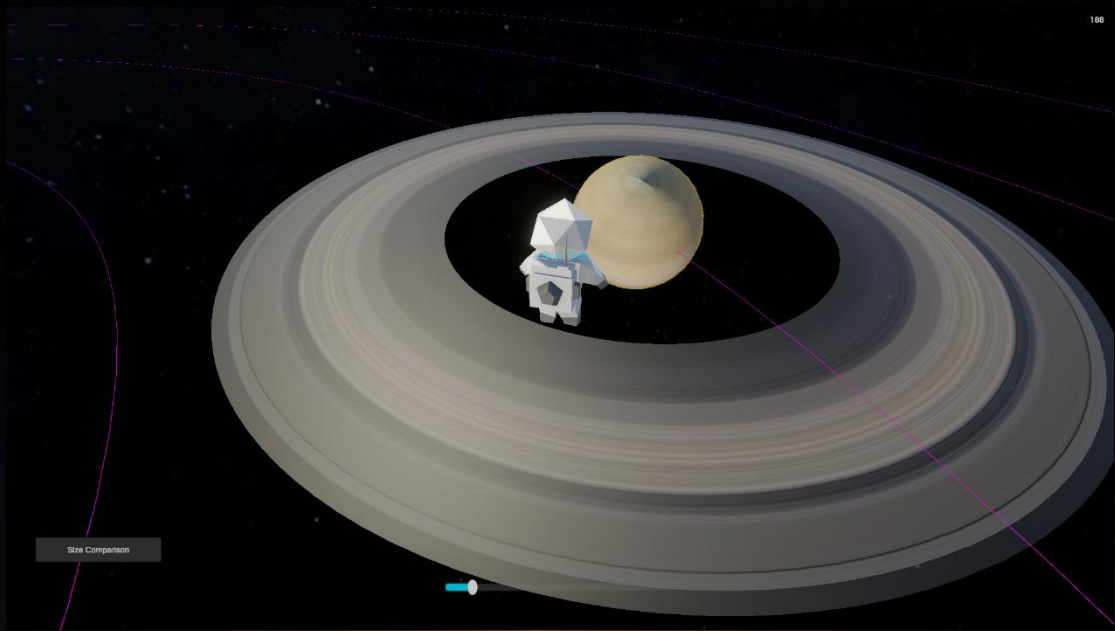
Feature 11 — Jupiter Simulation

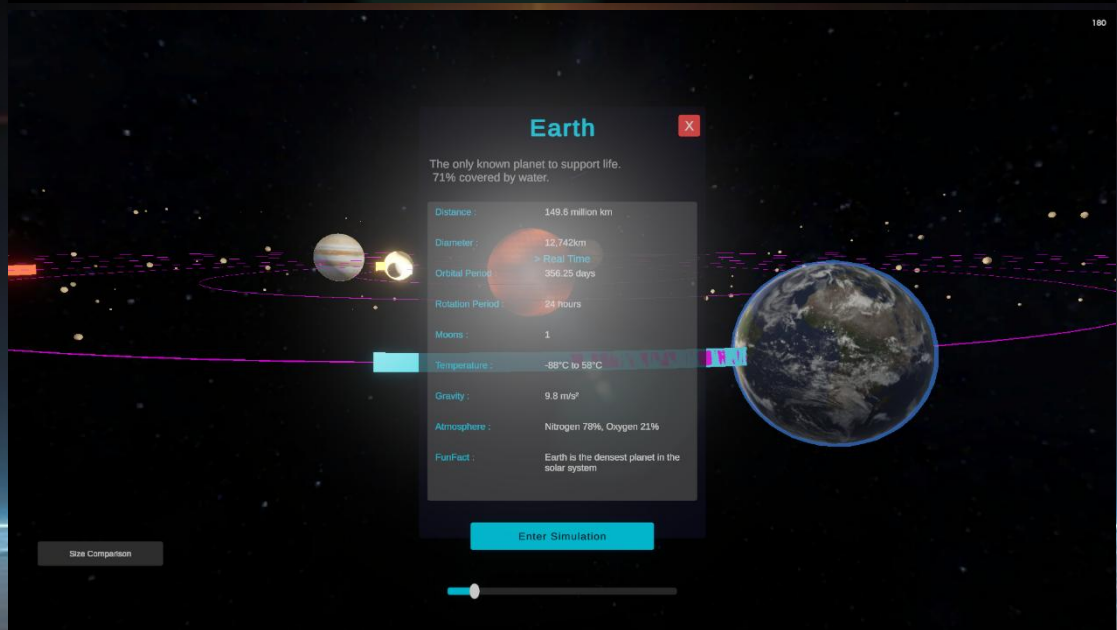
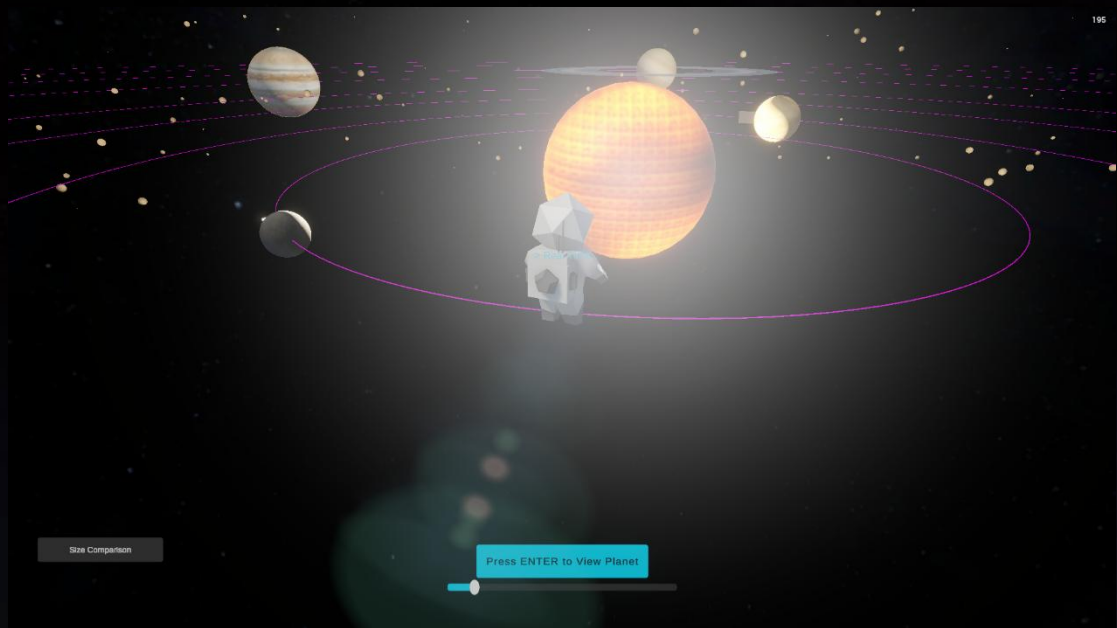
[Team Member 9 writes this — describe what their scene does: Great Red Spot, cloud bands, lightning, Galilean moons, UI data]



Screenshots









SOLAR SYSTEM

Interactive Simulation

EXPLORE UNIVERSE

QUIT

Back

515

SURFACE VIEW



JUPITER STATS

STORM: ACTIVE
WIND: 659 m/s
MOONS: 4 (GALLILEAN)
ROTATION: 9.93 HOURS

PRESS SPACE
TO FLY INSIDE



Code Structure Explanation

The project follows a component-based architecture where each script has a single responsibility:

Scripts/Core/ contains all gameplay logic:

- PlanetOrbit.cs handles orbital movement math
- PlanetRotation.cs handles self-rotation and tilt
- PlanetData.cs is a ScriptableObject data container
- PlanetClickHandler.cs detects mouse interaction
- SolarSystemManager.cs coordinates all subsystems
- CameraController.cs manages all camera behavior
- TimeController.cs manages simulation speed globally
- RingGenerator.cs creates Saturn rings procedurally
- SceneLoader.cs handles scene transitions
- CometOrbit.cs animates the comet elliptical path
- SpeedOfLight.cs spawns expanding light pulses

Scripts/UI/ contains interface logic:

- UIManager.cs controls all UI panels and text fields

Scripts/Audio/ contains sound management:

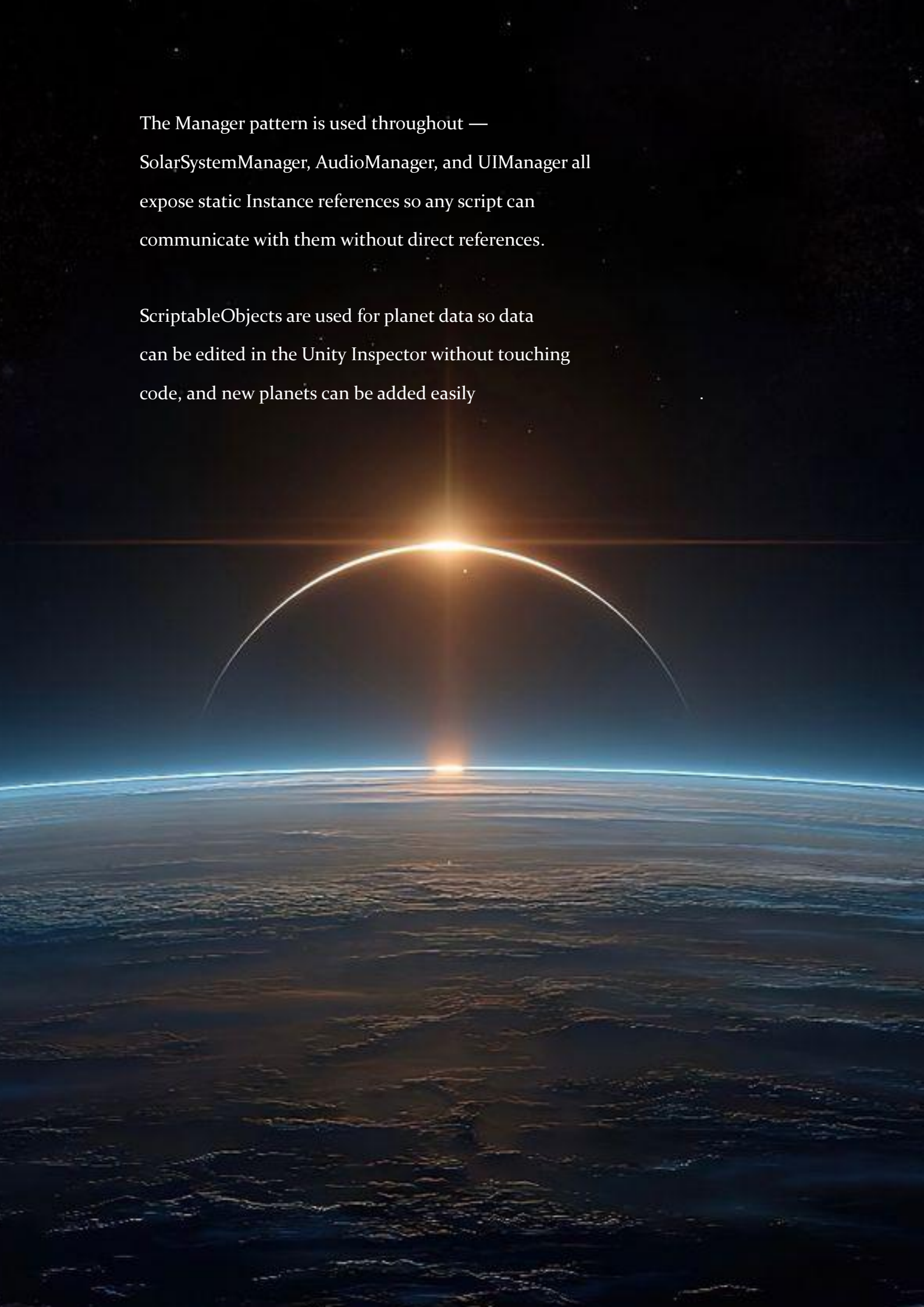
- AudioManager.cs handles music and planet sounds

Shaders/ contains custom visual effects:

- AtmosphereShader.shader creates rim glow effect
- SunShaderGraph creates animated fire surface

The Manager pattern is used throughout — SolarSystemManager, AudioManager, and UIManager all expose static Instance references so any script can communicate with them without direct references.

ScriptableObjects are used for planet data so data can be edited in the Unity Inspector without touching code, and new planets can be added easily



Challenges and Solutions

Challenge 1 — Planet Clicks Not Registering

Problem: Unity 6 uses the New Input System by default, which makes `OnMouseDown()` not work.

Solution: Replaced `OnMouseDown` with `Physics.Raycast` in `Update()` with a UI layer mask to ignore UI elements. Also added Physics Raycaster to the Main Camera component.

Challenge 2 — Canvas Blocking 3D Clicks

Problem: The UI Canvas was intercepting all mouse clicks before they reached the 3D planet colliders.

Solution: Set Graphic Raycaster Blocking Mask to Nothing on the Canvas, and disabled Raycast Target on all Image components inside the Canvas.

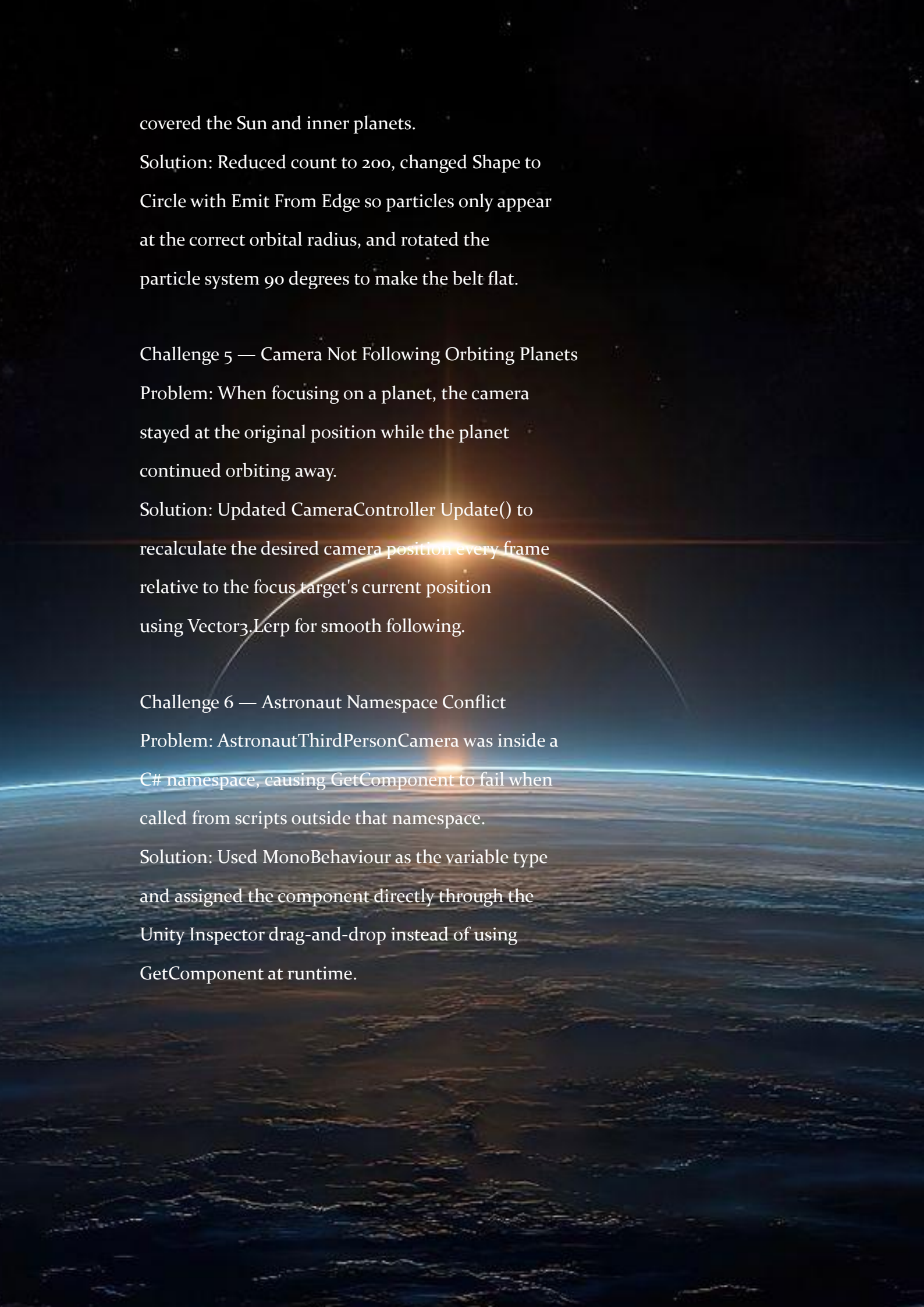
Challenge 3 — Unity Crashing on Font Generation

Problem: Generating the Orbitron font atlas at 2048x2048 resolution caused Unity to crash due to insufficient RAM during the process.

Solution: Reduced atlas resolution to 512x512 for generation then upgraded after freeing RAM by closing other applications.

Challenge 4 — Asteroid Belt Too Dense

Problem: The initial particle burst of 800 asteroids



covered the Sun and inner planets.

Solution: Reduced count to 200, changed Shape to Circle with Emit From Edge so particles only appear at the correct orbital radius, and rotated the particle system 90 degrees to make the belt flat.

Challenge 5 — Camera Not Following Orbiting Planets

Problem: When focusing on a planet, the camera stayed at the original position while the planet continued orbiting away.

Solution: Updated CameraController Update() to recalculate the desired camera position every frame relative to the focus target's current position using Vector3.Lerp for smooth following.

Challenge 6 — Astronaut Namespace Conflict

Problem: AstronautThirdPersonCamera was inside a C# namespace, causing GetComponent to fail when called from scripts outside that namespace.

Solution: Used MonoBehaviour as the variable type and assigned the component directly through the Unity Inspector drag-and-drop instead of using GetComponent at runtime.

Challenge 7 — Planets Glowing Like Stars

Problem: All planet materials had Emission enabled which made them appear as bright light sources rather than reflecting sunlight.

Solution: Disabled Emission on all planet materials except Earth where a low-intensity night lights emission map was kept to show city lights.



Conclusion and Future Work

Conclusion:

This project successfully demonstrates the application of computer graphics principles in a real-time interactive simulation. Using Unity 6 with URP, we implemented advanced rendering techniques including custom HLSL shaders for atmospheric effects, Shader Graph for the animated Sun surface, procedural mesh generation for Saturn's ring system, and physically-based materials with NASA texture maps for all eight planets.

The simulation achieves its primary goal of creating an educational and visually impressive representation of our solar system. Players can interact with every planet, view accurate scientific data, and experience the scale and beauty of space through both a free-roam camera and a third-person astronaut character.

The team successfully divided responsibilities across nine members, with dedicated simulation scenes for Mercury and Jupiter demonstrating the extensibility of the scene transition architecture.

Future Work:

- Add simulation scenes for all remaining 6 planets
- Implement gravitational physics anti-gravity mode where planets interact with real Newton gravity
- Add VR support using Unity's XR Toolkit
- Include constellation overlay on the skybox
- Add real-time planet size comparison mode
- Implement multiplayer support so multiple astronauts can explore simultaneously
- Add procedural planet generation for exoplanets
- Improve astronaut animations with custom rigged swimming motion clips
- Add educational quiz mode triggered when viewing planet data panels

